Project Summary

PESOSE: Track 3: RedTwin: AI-Enabled Digital Twins for Continuous Red Teaming of Open-Source Ecosystems

Overview

Assessing the security posture of large computing infrastructures is notoriously difficult. These are complex systems, dynamic in their connectivity and usage patterns, and they often carry out processes that must not be disrupted. Performing security analysis directly on a production system can cause problems, as scans introduce overhead and analyzing services for vulnerabilities can trigger crashes. Because of these risks, a security assessment is usually performed manually, sporadically, and in an ad hoc fashion by scarce human experts, while adversaries, now increasingly augmented with AI, operate continuously. The goal of this proposal is the development of RedTwin, an agentic framework for the continuous, in-depth, and non-disruptive security assessment of computing infrastructures built on open-source software. Such infrastructures admit a different approach, because the openness of the software enables the faithful replication of an infrastructure's topology and service configuration. RedTwin uses cooperating teams of AI agents to capture the configuration of a target system, observe its execution, build a *digital twin* of the system, run red team exercises against the twin instead of the production system, and, when vulnerabilities are found, guide a human-driven verification and remediation of the issues in the original infrastructure. Because the analysis runs on the twin, it can be repeated continuously and adapted as the target evolves.

The ecosystem that this project targets is anchored on two existing, widely deployed open-source products. The first is **nginx**, the reverse proxy through which traffic enters and moves within nearly every open-source deployment, and which we will harden against the memory-safety, configuration, and request-smuggling defect classes that dominate its history. The second is **ZMap**, the scanner with which operators discover what is actually running, and which we will extend into an instrument for safe, budgeted discovery inside production networks. Every finding that we confirm on a twin is driven upstream to the project responsible, and the work therefore improves not only individual deployments, but also the shared ecosystem on which they are built.

Intellectual Merit

The RedTwin framework contributes new agentic workflows that automate the tedious tasks of capturing an infrastructure's configuration and reproducing it in a virtualized environment. These workflows are supported by novel techniques for replicating large-scale infrastructure at reduced scale while preserving its security-relevant properties, and for measuring the representativeness of the resulting twin through logic-based invariants and attack-graph preservation analysis. The framework quantifies the fidelity of the twin: each analysis reports which security-relevant properties the replica preserves, together with a measure of how much of the target system the capture process resolved. Vulnerability-analysis agents then examine the twin using novel techniques for chaining vulnerabilities across a networked infrastructure, extending the investigators' prior work on AI-planning-based privilege-escalation chains and on a universal theory of in-program exploitation to complete open-source ecosystems.

Broader Impacts

Open-source software underpins the Nation's critical infrastructure, from health care and financial systems to utilities and government services. By enabling continuous, non-disruptive security assessment of the ecosystems built on this software, RedTwin will improve the resiliency of that infrastructure and help prevent attacks with catastrophic consequences. The improvements that the project contributes to nginx and ZMap reach every downstream user of those products, including the many organizations that cannot afford commercial alternatives. The project will also produce open-source artifacts and a public corpus of reference infrastructures, integrate infrastructure-security modules into the curricula at UC Santa Barbara and Arizona State University, involve undergraduate students in research, and organize a new public capture-the-flag competition centered on the creation of digital twins.

Keywords: critical infrastructure; open-source security; digital twins; AI agents; vulnerability analysis.

Project Description
RedTwin: AI-Enabled Digital Twins for Continuous Red Teaming of Open-Source Ecosystems

1 Introduction

Assessing the security posture of large computing infrastructures is notoriously difficult because they are complex systems, dynamic in their connectivity and usage patterns, and they often carry out processes that should not be disrupted. Performing security analysis directly on a production system can cause problems: a single aggressive probe against a database replica, a message broker, or an authentication service can cascade into an outage that violates a regulated availability requirement, corrupts state, or interrupts a process on which people depend.

Because of these risks, the analysis of the security posture of an infrastructure is usually performed manually and in an ad hoc fashion, by analysts whose expertise is scarce and expensive. As a result, these assessments are carried out sporadically, and the evolution of the network topology, the introduction of a new service, or a change in the configuration of a component might introduce a vulnerability that invalidates the results of the previous analysis. In the meantime, adversaries that are increasingly augmented with AI operate continuously, automatically, and at scale [28]. The proposed research targets this widening gap.

The goal of the proposed research is to develop a new agentic framework, called RedTwin, to support the continuous assessment of the security posture of computing infrastructures based on open-source software. RedTwin uses cooperating teams of AI agents to capture the configuration of a target system, monitor its execution, create a *digital twin* of the system, run red team exercises against the twin, and guide a human-driven verification and remediation of the issues that are found in the original system. The core principle of RedTwin is a strict separation between *observation*, which is passive, non-disruptive, and the only activity that ever touches the production system, and *analysis*, which is aggressive, potentially destructive, and confined entirely to the digital twin. This approach is feasible because the components of the infrastructure are open source, and therefore their behavior, configuration surface, and version history are transparent and inspectable. This transparency makes it possible to run a functionally equivalent instance of a component in a controlled environment, something generally infeasible for closed, proprietary appliances.

A motivating scenario. Consider a regional health-care provider whose electronic health-record system runs on a stack of open-source components: a PostgreSQL [48] cluster with streaming replication, a set of application servers behind an nginx [41] reverse proxy, a Keycloak [33] identity provider federated with the hospital directory, a RabbitMQ [52] message broker connecting clinical subsystems, and a fleet of medical-device gateways that speak proprietary protocols. This deployment is subject to strict availability and privacy requirements, and the security team cannot fuzz the application servers, cannot flood the broker with malformed messages, and cannot run an exploit against the identity provider. As a result, the questions that matter to the security team remain unanswered. Can an attacker who compromises a single device gateway pivot through the broker to reach patient records? Does the proxy configuration expose an internal administrative endpoint through a path that the access rules did not anticipate? Each of these questions concerns the *deployment* as a whole, and none can be answered from an advisory or a source repository. RedTwin is designed for this situation: it reconstructs the deployment as a digital twin, exercises exactly these attack paths against the twin, and reports the confirmed chains to the security team for verification on the production system, all without ever sending a single aggressive packet to the live infrastructure.

A possible alternative to a digital twin would be to leverage the staging environment that engineers already use to test applications before deployment. In practice, this would not be sufficient, as staging environments are built and maintained manually, they drift from production as the infrastructure evolves, and they are typically provisioned for functional testing rather than to preserve the security-relevant structure of the deployment: the exact component versions, the network reachability relations, the trust and authentication dependencies, and the data that determines authorization decisions. A staging environment that omits the device gateways, or that seeds an empty user directory, will not exhibit the attack paths exploitable in the real infrastructure. RedTwin constructs a digital twin whose fidelity is measured and controlled with respect to

explicit security properties, as described in Section 5.2, and that is kept synchronized with the production system by a continuous capture-and-observe loop, as described in Section 5.1.

Alignment with PESOSE Track 3 and contributions. This project is submitted to NSF 26-506, Track 3, which addresses vulnerabilities in open-source products and infrastructure, including both technical risks and socio-technical ones. The ecosystem we target is anchored on two existing, widely deployed open-source products, described in Section 2. The first is **nginx** [41, 42], which we will harden against the memory-safety, configuration, and request-smuggling defect classes catalogued in Section 4. The second is **ZMap** [25, 76], which we will extend into an instrument for safe, budgeted, and information-rich discovery inside production networks. Every finding that we confirm is driven upstream, so the work improves both individual deployments and the shared ecosystem on which they are built.

The project contributes: (i) agentic workflows that capture the configuration of an infrastructure into a continuously maintained knowledge base, together with upstream extensions that make open-source discovery safe on production networks; (ii) techniques for replicating large infrastructures at reduced scale, supported by a *quantitative fidelity measure*; (iii) methods for chaining vulnerabilities across a networked infrastructure, extending our prior work on AI-planning-based privilege escalation and on a universal theory of in-program exploitation to the setting of complete ecosystems; and (iv) the hardening of nginx and ZMap, a public corpus of reference infrastructures with ground truth, and an open-source release of the framework.

2 The Target Open-Source Ecosystem

Track 3 asks proposers to improve the safety, security, and privacy of an *existing* open-source ecosystem. This project targets the ecosystem that forms the observable and defensible perimeter of nearly every open-source deployment, anchored on two widely used, publicly available products: **nginx** [41, 42], through which traffic enters and moves within such deployments, and **ZMap** [25, 76], with which operators discover what is actually running. RedTwin hardens the first, extends the second, and uses both as the concrete ecosystem against which its methods are developed and evaluated.

nginx: the trust boundary of open-source infrastructure. nginx is an HTTP server, reverse proxy, load balancer, and TLS terminator that has been released under the two-clause BSD license and that has been in continuous development since 2004 [41]. It ranks consistently among the most widely deployed web servers in the world. The *development and testing model* follows a maintainer-review process around a public repository [42] and a development mailing list, with periodic mainline and stable branches, an in-tree functional test suite, continuous integration across a broad matrix of platforms and build configurations, and downstream testing by the distributions that package it. *Dissemination* occurs through official binary packages, distribution repositories, and container base images. The *user base* spans essentially every sector that runs open-source infrastructure, including the health-care, financial, utility, and government deployments that this proposal targets. The *contributor base*, in contrast, is comparatively small, and it consists of a core maintainer group, a set of occasional patch contributors, and a large third-party module ecosystem that is maintained outside the project. This asymmetry between the size of the user base and the size of the maintainer base is the structural risk that Track 3 targets.

ZMap: discovering what is actually running. ZMap is a single-packet, stateless network scanner that has been released under the Apache 2.0 license and that is maintained by an academic and industrial consortium [25, 76]. Its stateless design allows a single commodity machine to sweep enormous address spaces far faster than connection-oriented scanners, and the companion ZGrab tool performs application-layer handshakes that identify service software and versions [74]. The *development and testing model* is a pull-request-and-review process conducted in the open on GitHub, with unit and integration tests that are run on every pull request and with tagged releases that are packaged for major platforms. *Dissemination* occurs through the source repository, distribution packages, and container images, and ZMap underpins commercial and non-profit internet-measurement services as well as a substantial body of peer-reviewed research. The *user base* includes network operators who perform asset inventory, incident responders, attack-surface-management services, and academic measurement groups, and the *contributor base* is an active but modest community centered on the maintaining institutions.

ZMap was designed for internet-wide measurement from outside a network, where the marginal effect of a probe on any single host is negligible. This project requires the opposite, that is, repeated discovery *inside* a production network whose hosts may be fragile medical, industrial, or embedded devices. The probe rate that makes ZMap valuable for internet-wide measurement is unsafe against such hosts. Section 5.1 describes the extensions we will contribute upstream to make ZMap a first-class instrument for safe, budgeted, and information-rich discovery on production infrastructure.

The ecosystem view. These two products bracket the problem that RedTwin addresses, as ZMap determines what an operator can *know* about an infrastructure without disturbing it, and nginx determines what an attacker can *reach* once inside it. Both sit within a broader ecosystem that RedTwin must also capture, replicate, and analyze, including PostgreSQL [48], Keycloak [33], RabbitMQ [52], and Istio [31]. Our working relationships with the maintainers and users of nginx and ZMap, documented in the accompanying letters of collaboration, give the project a direct path from a finding produced on a twin to a fix that reaches every downstream user.

3 Societal, National, and Economic Impact

nginx and ZMap sit beneath a large fraction of the open-source software deployed today, and their defects are therefore inherited by every deployment built on them.

What depends on nginx. nginx sits at the point where untrusted traffic first meets an organization's software. It terminates TLS, enforces the first layer of access control, routes requests to the application servers, and mediates between the internet and the internal identity providers, brokers, and databases. A defect or a misconfiguration at that point is rarely contained, because it is a defect in the boundary that every other control assumes to be intact. Hospitals, payment processors, utilities, and government services all inherit this exposure, generally without any ability to audit it themselves.

What depends on ZMap. ZMap and the tools built on it are how defenders learn what they actually operate. Attack-surface management, triage after a new advisory, incident response, and regulatory asset inventory all reduce to the question of what is running, where, and at what version. When that question is answered incompletely, every downstream decision inherits the gap, because hosts absent from the inventory are not patched when an advisory lands. Improving the safety and the information yield of open-source discovery therefore improves the security of every organization that relies on it, and it disproportionately helps those that cannot afford commercial alternatives.

Economic and national significance. These components are deployed across the sectors on which the Nation depends, and the open-source stack as a whole has recently been valued in the trillions of dollars annually. The maintainer asymmetry described in Section 2 means that public investment in the security of these specific components generates returns far out of proportion to its cost: every vulnerability that RedTwin finds and drives upstream is removed simultaneously from every deployment that shares the affected component.

Measures of success. The project advances several of the measures of success named in the solicitation. It establishes new *data sets*, in the reference infrastructures, their ground-truth knowledge graphs, and the seeded vulnerability corpus of Section 7; *new technologies and techniques*, in the capture, replication, fidelity-measurement, and exploitation-chaining methods of Thrusts 1–3, all released as open source; and new *research infrastructure*, in the twin-construction toolchain and the public twinCTF infrastructure. Finally, it prepares participants for careers in *STEM* through the activities of Section 11.

4 Targeted Vulnerabilities and Risks

Memory-safety defects in C network services. nginx is written in C, and it parses attacker-controlled input at the network edge. Its history contains the defect classes that this combination implies. Examples are a stack buffer overflow that is reachable through chunked transfer encoding [15], an off-by-one heap write in the DNS resolver that was remotely triggerable across a twelve-year span of versions [17], and repeated memory-disclosure defects in optional modules that many operators did not know they had compiled in [19].

The component-level analysis of Section 5.3 targets these defects, and because that analysis is destructive it cannot be run against production.

Configuration-induced vulnerabilities. The larger practical risk is not a flaw in nginx, but a flaw in the way that nginx is configured. The directive language admits well-documented footguns whose consequences are severe and whose presence is invisible to the operator. For example, an `alias` directive without a trailing slash yields path traversal, disabling slash merging in front of an upstream that normalizes differently yields an access-control bypass, a `proxy_pass` directive that uses a variable silently disables URI normalization, and parsing mismatches between nginx and its upstream enable request smuggling [16]. This class of defect is systemic, and it has been shown to generalize across proxies and application servers [70]. These vulnerabilities exist only in the *deployed configuration*, and cannot be found by analyzing the nginx source alone, nor in a staging environment that does not reproduce production. Detecting them therefore requires a replica of the deployed configuration, which is the replication problem addressed in Section 5.2.

Socio-technical risks. Deployments do not install nginx. They install a container image, a distribution package, or an ingress controller, such as `ingress-nginx`, that contains some build of nginx, with some set of modules, at some version, and from some registry. The Log4Shell incident showed how a defect in a transitive dependency propagates instantly across every system that unknowingly embeds it [18]. RedTwin records provenance and Software Bills of Materials in the knowledge base, as described in Section 5.1, so that the analysis can reason about attack paths that originate in a dependency which the operator never chose explicitly. In doing so, we follow the recommended practices of CISA and NSA [12] and the criteria of the OpenSSF [43]. The *third-party module* ecosystem of nginx is maintained outside the core project and receives a fraction of its scrutiny, yet modules are compiled into the same address space with the same privileges, and an unmaintained module accumulates defects for which no advisory is ever issued to prompt an update. A second, related risk is the concentration of *maintainer and governance* authority, which is itself a security property: the xz-utils backdoor showed that a patient adversary can obtain maintainer trust and introduce a backdoor that survives ordinary review [20].

Prior incidents and threat model. The incidents that instantiate these classes share a structure that motivates the design of this project. The clearest example is the set of defects that were disclosed in 2025 in the `ingress-nginx` admission controller, collectively named "IngressNightmare," which permitted unauthenticated remote code execution and the disclosure of cluster-wide secrets on a large fraction of Kubernetes deployments [21]. Whether a given cluster was exploitable depended on the network policy, on whether the controller was reachable from the workload pods, and on the surrounding configuration, so reachability mattered as much as the defect itself. Answering that question requires a faithful replica, since the probing it demands cannot safely be performed against production. The same structure recurs in the resolver defect of 2021, whose exploitability depended on whether an attacker could influence DNS responses from the network position of the deployment [17], and in request smuggling, which depends on the specific pairing of proxy and upstream [16, 70]. In each of these cases the advisory establishes that a defect exists, while exploitability depends on deployment-specific configuration that no advisory records.

Accordingly, we consider an adversary whose goal is unauthorized access to a designated high-value asset, such as a patient database, a credential store, or a cluster control plane. We model four initial positions: an *external* adversary, who has network access only to internet-facing services; a *foothold* adversary, who has compromised one low-privilege component; an *insider* adversary, who holds valid credentials for one role; and a *supply-chain* adversary, who controls one dependency or build artifact. The adversary is assumed to know the open-source composition of the infrastructure, since that information is public, but not its deployment-specific configuration or its secrets. Two assumptions bound the work. First, RedTwin evaluates the security of the *deployment*, and physical attacks, social engineering, and the compromise of the virtualization platform are out of scope. Second, RedTwin never attacks the production system, and any conclusion about production is therefore bounded by the measured fidelity of the twin, which Section 5.2 defines and quantifies.

5 Technical Approach and Research Thrusts

RedTwin creates a digital twin of an infrastructure, performs security tests on the twin, and uses the results to improve the security posture of the original. Three challenges shape the design. First, the configuration of the original must be captured continuously, as complex infrastructures are in constant evolution. Second, most large infrastructures cannot be fully replicated, and the twin must be smaller in scale but still representative. Third, as the infrastructure evolves, the twin has to evolve as well, or the results will describe a system that no longer exists.

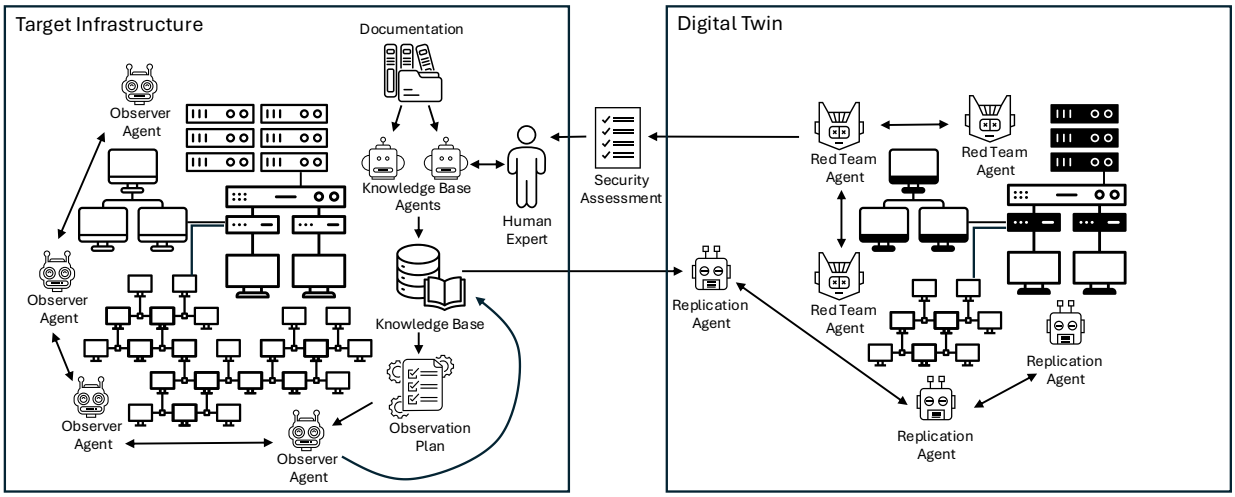

Figure 1: Overview of the RedTwin framework. Knowledge Base Agents combine documentation with human expertise to build a Knowledge Base, driving an Observation Plan that deploys Observer Agents, which collect dynamic information without disrupting operations and feed it back. Replication Agents construct a digital twin from the Knowledge Base, and Red Team Agents continuously assess that twin, producing findings that are verified and remediated on the target infrastructure.

RedTwin addresses these challenges by leveraging four cooperating teams of agents, summarized in Figure 1 and developed in the thrusts below. The Knowledge Base Agents build and maintain the model of the target, the Observer Agents enrich that model by watching the running system without ever touching it, the Replication Agents construct the twin, and the Red Team Agents attack the twin and report their findings to a human analyst. When a finding is a false positive, or an artifact of an imprecise replication, the information flows back to the Knowledge Base and Observer Agents, so each analysis cycle also improves the fidelity of the twin. In the health-care deployment of Section 1, this loop is what turns an undocumented administrative endpoint, a permissive `proxy_pass` rule, and a deserialization flaw in a broker client library into a single reported chain from a device gateway to the patient database.

5.1 Thrust 1: Capturing Open-Source Infrastructure Configurations

Knowledge base design. Structured data will be used to represent the network topology and the roles of the various components. For this layer, we will use a graph database, such as Neo4j [40], whose model is a natural fit for the questions that RedTwin must answer, such as which hosts can reach a given service, which services share a credential store, and which paths connect an externally exposed component to a sensitive asset, while a vector store, such as Qdrant [50] or Weaviate [71], will hold the natural-language documentation. A research challenge in this step is the design of a schema expressive enough to capture the security-relevant structure of an arbitrary deployment, yet constrained enough to support the queries and invariants used downstream. We will model hosts, containers, services, network segments, credentials, data stores, and trust relationships as typed nodes and edges, and we will attach to each of them the *provenance* of the information together with a *confidence* level. Both are essential, because the same fact may be asserted by a configuration manual and contradicted by a runtime observation. Version information must be fine enough

to map a deployed component to the package, release, and build options that determine its vulnerabilities, since the component name alone does not identify a vulnerability set, whereas the version, the module set, and the directive configuration do.

Observation techniques. The Observer Agents monitor the system statically, through documentation, code, configuration files, and infrastructure-as-code artifacts, and dynamically, through the runtime observation of network and host activities. Examples of these techniques are passive network analysis at key north-south and east-west points, the interpretation of logs into a shared ontology by means of LLMs, API monitoring through facilities such as eBPF or service meshes like Istio [31], configuration monitoring through managers such as Ansible [4] or Puppet [49], and the ingestion of alerts from IDS/IPS, EDR, or SIEM tools. In addition, we plan to extend our previous work on service security dependency analysis [72]. That research demonstrated that by observing the dynamic behavior of services it is possible to derive their security properties and dependencies. Accurately modeling these dependencies is essential: any dependency missed during capture is absent from the twin and therefore never exercised by the analysis.

Safe active discovery: extending ZMap. Passive observation cannot resolve every question, because whether a service is reachable from a given segment, and what version it runs, is often answerable only by sending a packet. The stateless design of ZMap [25] makes it the right foundation for this task, but its assumption that the marginal effect of a probe is negligible does not hold inside a hospital or a plant. We therefore plan to develop, and contribute upstream, four extensions: *impact-budgeted scheduling*, that imposes per-target budgets on probe rate and concurrency derived from what the knowledge base records each host to be, with hard interlocks that abort a sweep when the observed latency or error rates degrade; *knowledge-graph-native output*, that emits typed assertions with provenance and confidence, so discovery feeds the schema directly; *richer safe fingerprinting*, that extends the ZGrab [74] handshakes to recover the version and module detail needed to map a service to an upstream package; and *incremental re-scan*, that prioritizes re-probing where confidence has decayed. These extensions address the reason why many operators decline to scan their own networks, and they are a concrete Track 3 improvement to a widely used product.

Observation planning and coverage. We will formulate the synthesis of the observation plan as a constrained optimization problem. Each candidate vantage point offers a certain amount of information gain, at a cost expressed in terms of the load on the production system, the intrusiveness of the deployment, and the consumption of scarce resources such as LLM queries, and the plan maximizes the coverage of the components most relevant to the security model while respecting hard constraints, for example, never deploying an agent on a host that carries a latency-sensitive clinical workload. Because observation is deliberately non-aggressive, the resulting model is necessarily incomplete. We will therefore develop a *model coverage* metric that estimates what fraction of the infrastructure the capture has resolved, using saturation analysis over observation time, cross-validation among documentation, infrastructure-as-code, and runtime observation, and, on the reference networks, direct measurement against withheld ground truth. Coverage is reported alongside every finding, and it bounds every claim RedTwin makes about production.

Supply chain and the human loop. In addition, we will incorporate SBOMs and package manifests into the capture process, so the knowledge base records not just what is running, but also where each component came from. This addresses the socio-technical risks described in Section 4, and it feeds the remediation loop of Thrust 3. Note that, while the process is presented here as a sequence, the agents will actually work continuously: new documentation triggers new observation plans, and observations inconsistent with the documentation are presented to the administrator for disambiguation. We envision this as an agent-human collaboration, because deploying observer agents modifies the infrastructure and must be vetted, but as the agents become more knowledgeable, we expect that the level of interaction will diminish, and Section 7 measures whether it does.

5.2 Thrust 2: Replicating Computing Infrastructures

The key problem in this thrust is how to build digital twins that are scale-reduced and yet still representative of the original. We organize the approach around three insights, and we then make faithfulness quantitative.

Insight 1: infrastructures replicate for load. If 200 file servers share the same operating-system image, package versions, and firewall rules, they can be considered equivalent from a security modeling perspective, and a single instance can stand for all of them. This reduces the veracity of the twin, because attacks that exploit resource bottlenecks will not transfer. The abstraction is therefore a trade-off between faithful replication and the minimization of resource usage, and the fidelity measure defined below reports where a given twin sits on it. We will formalize this idea as an equivalence relation over the components of the infrastructure. Two components are placed in the same class when they agree on the attributes that determine their security behavior, such as the software image and version, the set of open ports and exposed services, the applicable firewall and access rules, the trust relationships in which they participate, and their position in the network topology. The twin then instantiates a single representative per class, together with a *multiplicity annotation*. A central research question is which attributes must be included to preserve a given class of attacks, and we will study it empirically, by measuring whether the attacks discovered on the abstracted twin still succeed when the abstraction is refined, and analytically, by relating the attributes to the preconditions of the attack primitives modeled in Thrust 3.

Insight 2: services operate in patterns. Network administrators tend to reuse combinations of components that work well and with which they are familiar, and a particular pairing of a proxy and an upstream recurs throughout an infrastructure. Each pattern can carry pattern-specific vulnerabilities, and the replication algorithm will identify these patterns and will make sure that each of them is instantiated in the twin. This matters directly for our anchor product: request smuggling is a property of a *pairing* and not of nginx alone, so a twin that collapses distinct pairings into one cannot exhibit those defects.

Insight 3: data is as relevant as code. If an LDAP server uses a specific database of users to authorize access to a host, that data determines whether an attack path exists. Organizations usually have to deal with three types of data. *Semantic data* represents the things the organization deals with, such as patients, diagnoses, and insurance claims. *Structural data* defines the shape of the production data systems, including schemas, foreign-key relationships, record counts and sizes, and, in the case of hospitals and other health-focused organizations, domain objects such as DICOM images or FHIR resources. *Operational data* reproduces how the infrastructure behaves, including login frequency, API request patterns, query mixes, and backup jobs. Specialized agents will profile the source environment for the aggregate properties required for fidelity, will generate a semantically consistent synthetic state, and will enforce invariants such as referential integrity, protocol semantics, and causal ordering. In the context of health-management infrastructures, such as hospitals, we plan to leverage existing open-source medical datasets to create realistic datasets. To this end, we include a letter of support from the CEO of Sage Bionetworks, an organization whose goal is to provide access to open-source medical and health-related datasets. Using the datasets managed by Sage Bionetworks as well as those that are brokered by that organization, we are planning to create high-fidelity synthetic datasets that will maintain the characteristics of real-world medical data.

Reconciling sanitization with authorization. Sanitization and the preservation of security behavior are in direct tension, because authorization outcomes frequently determine whether an attack path exists. For example, replacing real users with synthetic ones can silently remove the group membership that made a privilege escalation possible. We will therefore specify the *authorization structure* separately from the *data content*. Identities, groups, roles, ACL entries, and the membership and delegation edges among them will be reproduced as an isomorphic graph that preserves the cardinalities and the shape of every permission relation, while the identifying content is replaced. Secrets become surrogates of equivalent strength and *distribution*, preserving the properties an attacker could exploit, such as the reuse of one credential across services. Determining which authorization-relevant properties must be preserved exactly, which only distributionally, and which can be discarded is a research question of this thrust.

Making fidelity quantitative. Once the twin has been created and the data instantiated, we must analyze its configuration with respect to the original infrastructure, and to this end we will extend graph- and metric-based comparison algorithms [68]. We plan to approach this problem in two ways. First, we will develop techniques to represent the properties of the network using logic-based approaches. These properties could

define access, for example, "service X is reachable from host Y", or topology, for example, "the cluster Z can only be accessed through host W", and they are then treated as invariants that must be preserved by the replication process. Determining what properties are important to represent the security posture of a network, and how they can be verified, is an integral part of the research, and it connects to work on network configuration verification [27, 6]. Second, we will perform an attack-graph preservation analysis. We will generate attack graphs [61, 45] for both the original infrastructure and the twin, and we will establish that the attack graphs of the twin are a sound abstraction of the attack graphs of the original. Building on these two techniques, we will define a fidelity measure that reports which security-relevant properties are preserved, which are approximated, and which are lost. The measure rests on two distinct notions. *Soundness* is the property that every attack path present in the original has a corresponding path in the twin. It is what the equivalence-class abstraction is constructed to preserve, and it either holds or it fails; the quantity we can measure against ground truth is *attack-path recall*, the fraction of the original's attack paths that a given twin reproduces. Recall below one therefore identifies a defect to be diagnosed and not a tolerance to be accepted: some abstraction decision discarded an attribute on which an attack depended, and the missing path identifies which one. *Precision* is the fraction of the paths in the twin that correspond to real paths in the original, and the abstraction trades precision away deliberately, so a Thrust 3 finding that does not reproduce likewise localizes to a specific abstraction decision that the Replication Agents can then refine.

What soundness can and cannot mean here. Soundness, as defined above, carries a specific limit. The attack graph of the original can only be derived from the captured model, itself the product of deliberately non-aggressive observation. What we can establish is therefore *soundness relative to the captured model*, and not an unconditional claim about the physical system. On the reference networks, where ground truth exists by construction, we will evaluate the stronger property, by comparing the attack graph of the twin against one derived from the true configuration, and on production every soundness claim is reported jointly with the model coverage metric of Thrust 1.

Components that cannot be replicated. For proprietary appliances, unavailable hardware, and closed-source services for which no substitute exists, the Replication Agents will synthesize a mock that reproduces the externally observable behavior recorded by the Observer Agents. Mocks introduce *false negatives* by construction: a mock reproduces only the behavior the Observer Agents recorded, and exploitable behavior is frequently behavior that was never exercised in production. We therefore treat every mocked component as a declared region of unsoundness. The fidelity report marks which parts of the attack surface are mock-backed, findings whose chains traverse a mock are flagged as requiring verification on the production system, and the coverage measure is reduced accordingly. Our previous research on re-hosting proprietary firmware [30, 13, 66, 59] provides the starting point: it demonstrated that static and dynamic analysis combined with runtime observation can execute proprietary firmware on emulators such as Qemu [51], even without documentation about the underlying hardware.

5.3 Thrust 3: Analyzing the Security Posture of Digital Twins

This thrust combines component-level techniques, such as fuzzing [26, 59], static analysis [35], and symbolic execution [55], with system-level attacks, such as multi-step breaches [54]. Our previous work in the DARPA AIxCC competition, in which we developed the Artiphishell system [60], has shown that the problem of identifying complex, multi-step vulnerabilities can be formulated effectively using agentic workflows. The Red Team Agents orchestrate these established techniques rather than reinventing them, selecting and configuring the appropriate tool for a given service and interpreting the results in the context of the knowledge base. The value added by the agentic layer is twofold: it maps a low-level defect to the role that the affected component plays in the infrastructure, and it expresses the defect as an exploitation primitive with preconditions and postconditions that the system-level planner can compose. Here the open-source nature of the ecosystem is an advantage: the source and the build configuration of each component are available, and the agents can analyze them with a depth that would be impossible for closed appliances.

Hardening nginx. nginx is the first target of this machinery, and it is the primary vehicle through which this project delivers Track 3 outcomes upstream. We will pursue this along three lines. For *memory safety*, we will

apply our fuzzing and static-analysis tooling [26, 35] to the request and response paths and to the optional modules that historically carry defects, running against twin-hosted instances configured exactly as they are configured in the deployments we capture, reaching configurations that the upstream test suites do not exercise. For *configuration-induced vulnerabilities*, we will use the record of real deployed directive configurations held in the knowledge base to build a checker that identifies the traversal, normalization, and access-bypass patterns of Section 4, and we will contribute the resulting rules as tooling that the wider community can run. For *request smuggling*, we will exercise the proxy-upstream *pairings* as they actually occur in captured deployments, differentially testing how nginx and each upstream parse the same request. Such defects are properties of the pair, and testing either component in isolation cannot reveal them. Confirmed defects are reported upstream under the process of Section 6. Where a defect is configuration-induced instead of a code flaw, the deliverable is detection tooling and documentation, since there is no patch to submit.

Composing multi-step attacks. The workflow will instantiate a clean version of the twin, and the attack planning will have access to the knowledge base, so that the planner reasons with the information available to the knowledgeable adversary of Section 4. The combination of single attacks will build on top of the model of preconditions and postconditions of attacks that we have explored in our ChainReactor [46] and UTE [24] research efforts. ChainReactor automated the discovery of privilege-escalation chains via AI planning. UTE, which we have developed as part of the DARPA HARDEN program, treats exploitation as a search problem: any exploitable bug exposes a small set of operations that an attacker can induce, for example allocations, frees, reads, writes, and file operations, and by modeling the effects and the preconditions of these operations abstractly, the construction of an exploit becomes tractable for a solver. Furthermore, this same abstraction allows us to reason about chains that cross theories, for example, a buffer overflow that exists only if the attacker can control the contents of a file. UTE has been designed for in-program analysis and ChainReactor for in-host analysis, and **extending and combining both of them to reason across a networked infrastructure is a core research contribution of this thrust.** The planning activity targets a designated high-value asset, and unauthorized access to that asset demonstrates a breach. To scale to infrastructures with many hosts, we will exploit the same equivalence-class structure used for replication, as an attack step feasible against one representative of a class is feasible against every member. This lets the planner reason at the level of classes and roles rather than individual hosts, and then lift a discovered chain into a family of concrete chains in the original.

What "continuous" costs. RedTwin operates on three cadences: *event-triggered* re-capture, within minutes of an infrastructure-as-code commit or of an observer detecting an undocumented host; *scheduled* re-analysis, nightly for the component-level analysis of changed components and weekly for the full multi-step planning; and *advisory-triggered* re-analysis, when a new CVE affects a component that the knowledge base records as deployed, reducing the question of whether the organization is exposed from a multi-day manual exercise to a query against a twin that already exists. The dominant recurring costs are LLM inference and the compute for twin instantiation and fuzzing. Establishing the cost per cycle, and reducing it through incremental re-capture and caching, is an explicit engineering objective, and Section 7 reports it as a metric.

From findings to prevention, detection, and recovery. Consistent with the goals of PESOSE Track 3, the output of this thrust is oriented toward prevention, detection, and recovery. The result of an analysis is a detailed document that describes the multi-step attack, and this document is communicated to a human security analyst, who will have to validate the findings on the actual system. False positives trigger a re-evaluation of the fidelity of the replication, so this thrust both produces actionable findings and improves the twin. Each confirmed chain is accompanied by the set of preconditions that enabled it, which directly suggests remediations, such as removing an unnecessary reachability relation, updating a component to a patched release, tightening an access-control rule, or segmenting a trust boundary. When a finding traces to a defect in an upstream component, the provenance captured in Thrust 1 identifies the project responsible, and the chain serves as a high-quality report. Furthermore, because RedTwin produces attack graphs, the same artifacts can be used to prioritize monitoring and to inform recovery planning, by identifying the choke points whose compromise would be most damaging.

6 Build and Test Infrastructure, Quality Control, and Disclosure

Build and test infrastructure. RedTwin is developed in a public monorepo, and every change is submitted as a pull request that must pass continuous integration before it is reviewed. The suite includes unit tests for the tooling of each agent team, integration tests that exercise capture, replication, and analysis end to end against a small reference infrastructure, and regression tests derived from every previously confirmed finding, so that a capability once demonstrated cannot be silently lost. Because the outputs are produced by non-deterministic agents, the integration suite executes each scenario multiple times and asserts on *distributions* of outcomes rather than on exact traces, and the run-to-run stability is tracked as a reported metric. The reference infrastructures are themselves infrastructure-as-code, versioned alongside the source, so any result can be reproduced from a commit hash.

Quality control and security of new content. Contributions require the review of a project maintainer, and they must pass automated checks before they are merged, including static analysis and linting, dependency scanning against advisory databases, and a verification that any new dependency is recorded in the SBOM of the project. Releases are tagged, signed, and published together with a generated SBOM and a provenance attestation, following the recommended practices of CISA and NSA [12] and targeting the best-practices criteria of the OpenSSF [43] and the build-provenance levels of SLSA [65]. Content generated by agents receives specific scrutiny. For example, twin configurations, mocks, and synthetic datasets each carry a manifest that records what produced them and with what confidence, and the synthetic data is validated by the constraint agents of Section 5.2 and checked to contain no material derived from production records. Twins are held in isolated networks, access to them is restricted to the project team and to the owning organization, and they are destroyed at the end of an engagement.

Licensing and upstream contribution. RedTwin will be released under a permissive OSI-approved license matching those of the anchor products (BSD-2-Clause for nginx, Apache 2.0 for ZMap), so contributions can flow upstream without license friction, and the reference infrastructures and their ground-truth knowledge graphs will be released under a permissive data license. Improvements to the anchor products are developed as upstream contributions from the start, following the process of each project rather than being maintained as a fork. For example, the ZMap extensions will be submitted as pull requests with tests [76], and the nginx findings will follow the nginx security process. Where a finding concerns a deployment configuration instead of component code, the deliverable is detection tooling and documentation contributed to the community.

Responsible disclosure and dual use. RedTwin automates the discovery of exploitable chains in widely deployed open-source software, and the resulting dual-use considerations shape the policies below. All findings follow coordinated vulnerability disclosure, in that a confirmed upstream defect is reported privately to the security contact of that project, together with a reproducer and a proposed remediation, and publication is withheld until a fix is available or until a standard embargo period has elapsed. Findings specific to the deployment of a partner belong to that partner. The assessment of any system is performed only under a written authorization from its owner, specifying the scope, the permitted observation techniques, and the rules of engagement. The campus evaluations of Section 7 are conducted under agreement with the responsible IT organizations at UCSB and ASU, with the impact interlocks of Section 5.1 enforced throughout, and we will seek IRB review where any activity touches human-subjects data. We will publish the framework, the reference infrastructures, and the methods in full, because withholding them would disadvantage defenders without impeding sophisticated adversaries, who already possess the capability we automate [28]. However, we will not publish weaponized exploits for unpatched defects, and the released artifacts require an authorization scope before an assessment can be run.

7 Evaluation Plan

Reference infrastructures. Evaluating capture and replication requires ground truth that production systems rarely provide, and we will construct a suite of reference networks for which the topology, the configuration, and the set of planted vulnerabilities are fully known to us. These networks will be built from widely deployed open-source components, including our anchor products and the databases, brokers, identity providers, and orchestrators that surround them, and they will be described entirely through infrastructure-as-code, so we can

instantiate them reproducibly, vary their scale, and generate the ground-truth knowledge graph automatically. We have developed substantial expertise in this domain through our work on the Global Accessible Test Infrastructure (GATE) as part of the ACTION NSF AI Institute [1], in which we have created topologies that represent smart cities, chemical plants, water systems, and manufacturing plants. We will seed each reference network with a mixture of known vulnerabilities that are drawn from public advisories, including the nginx defect classes of Section 4, and with deliberately introduced misconfigurations. In addition, we will design a subset of the networks to require genuine multi-step chains, so success cannot be achieved by finding a single isolated flaw. These reference networks will also serve as the challenges for the twinCTF competition described in Section 11, and they will be released as a public benchmark.

Metrics and targets. Targets are stated for the reference networks, where ground truth exists.

Metric	**Target**
Thrust 1 — capture	
Hosts and services correctly identified	$\geq 95\%$ of ground truth
Service dependencies correctly identified	$\geq 90\%$
Model coverage estimate vs. measured	within 10 points
Expert input per 100 hosts	≤ 4 h, declining across cycles
Production impact during observation	no measurable latency or error change
Thrust 2 — replication	
Attack-path recall (paths preserved)	$\geq 90\%$
Precision (twin paths real in original)	$\geq 75\%$
Resource reduction vs. full replica	$\geq 10\times$ at equal recall
Thrust 3 — analysis	
Seeded single-component defects found	$\geq 85\%$
Seeded multi-step chains found	$\geq 70\%$
Findings failing to reproduce on original	$\leq 20\%$
Run-to-run stability, identical inputs	≥ 0.8 Jaccard overlap
Cost per assessment cycle (100 hosts)	reported; reduced $\geq 2\times$
Confirmed defects reported upstream	≥ 10 across nginx, ZMap, deps

We regard the upstream-report target as the most important line in this table, since a fix that lands upstream reaches every deployment of the affected component.

Baselines and ablations. Where established baselines exist, we will compare against them. For component-level analysis, we will compare the defects found by RedTwin against those found by running the underlying fuzzing, static-analysis, and symbolic-execution tools in isolation, in order to quantify the value added by the agentic orchestration and by the context of the knowledge base. For capture, we will compare against an unmodified ZMap [25] and against inventory tools such as osquery [44] and cartography [10], measuring both coverage and production impact, and, for configuration properties, against network configuration verification [27]. For multi-step analysis, we will compare against our own ChainReactor [46] on the single-host problems for which it was designed, and against attack-graph generation [45] on networked problems. Throughout, we will report the human effort required at each stage, since a central claim of the project is that RedTwin shifts experts from manual assessment to the verification of high-confidence findings.

Validation with existing users. Reference networks establish correctness against ground truth; whether the results are useful can only be determined on real deployments. Working with the collaborators whose letters accompany this proposal, and with industry and critical-infrastructure partners, we will evaluate RedTwin on operational systems in the later phase of the project. The ZMap extensions of Section 5.1 will be validated by operators who run them on their own networks and who report coverage and impact, and the nginx configuration checks of Section 5.3 will be validated against real deployed configurations contributed by partners. Subject to written authorization, as described in Section 6, we will apply the full framework to the internal research networks at UCSB and ASU and to partner systems. In these settings, the emphasis of the evaluation shifts to the non-disruptiveness of the observation phase, to the accuracy of the continuously maintained model as the infrastructure changes, and to whether a finding changed what the administrators who own the systems actually did.

Robustness to the evolution of language models. Because the agents interact only through typed interfaces, models are swappable, and we will evaluate open-weight models alongside frontier ones, so that the framework does not depend on any single provider. The reference networks carry ground truth, which lets us hold the benchmark fixed, substitute the model, and report the difference. We expect the model-independent components of the contribution, namely the knowledge-base schema, the equivalence-class abstraction, the fidelity measure, and the separation of passive observation from aggressive analysis, to remain valid as model capability improves, and the substitution experiment measures whether they do.

8 Milestones and Timeline

Although the focus of each thrust is sequenced, we plan to operate around a working end-to-end prototype from the first quarter, so that every component is exercised in context throughout the project. The targets below refer to the metrics of Section 7, and each year ends with a go/no-go review.

Qtr	Deliverable	Success criterion
Q1–Q2	Knowledge-base schema with provenance and confidence; documentation-ingestion agents; two reference networks (10–20 hosts) as infrastructure-as-code; Observer Agents and passive observation; service-dependency capture; first ZMap safety extensions (impact-budgeted scheduling, interlocks).	Schema expresses all reference-network ground-truth entities; $\geq 85\%$ of hosts and services recovered; no measurable production impact under interlocks.
Q3–Q4	Observation planning as constrained optimization; model coverage metric; knowledge-graph-native ZMap output and safe fingerprinting; first-generation Replication Agents (equivalence-class abstraction, interaction patterns, synthetic data with authorization-structure preservation). **First upstream ZMap contribution; end-to-end prototype.**	$\geq 95\%$ hosts/services and $\geq 90\%$ dependencies; coverage estimate within 10 points; 20-host twin built unattended at $\geq 5\times$ resource reduction. **Go/no-go:** end-to-end run with no manual intervention between stages.
Q5–Q6	Logic-based invariants and attack-graph preservation; quantitative fidelity measure; mock synthesis with declared unsoundness regions; component-level Red Team Agents; **nginx hardening campaign** (memory-safety analysis, configuration-defect checker, differential proxy-upstream smuggling tests).	$\geq 90\%$ attack-path recall and $\geq 75\%$ precision; mock-backed surface correctly identified; $\geq 85\%$ of seeded single-component defects; **first confirmed defect reported upstream**.
Q7–Q8	Multi-step planning (ChainReactor and UTE extended to networks; equivalence-class lifting); analyst verification workflow; continuous-operation cadences and cost reduction; evaluation on partner and campus networks; public release of framework, benchmark, and twinCTF challenges.	$\geq 70\%$ of seeded multi-step chains; planner scales to 100 hosts; $\leq 20\%$ of findings fail to reproduce; ≥ 10 upstream reports total; artifacts reproducible from published commits.

The ZMap extensions gate the quality of the capture that the first-generation Replication Agents consume, so they are scheduled across Q1–Q4, with acceptance criteria stated in terms of downstream usability instead of feature completeness. The fidelity measure gates the interpretation of every result of Thrust 3, and it therefore precedes the analysis milestones in Q5–Q6. Should that milestone miss its recall target, the remaining quarters will proceed on the reference networks, where ground truth substitutes for the measure, while the work on fidelity continues in parallel.

9 Related Work

Breach and attack simulation. The closest operational analogue to RedTwin is continuous adversary emulation, exemplified by MITRE CALDERA [9] and by a commercial breach-and-attack-simulation category. These systems run *against production*. No fidelity question therefore arises, but the same choice restricts them to behaviors known to be safe, because a false step damages a live system. They replay catalogued techniques, and they do not fuzz a parser or exploit an unknown defect. RedTwin makes the opposite trade, accepting a fidelity obligation, which Section 5.2 discharges quantitatively, in exchange for the freedom to run destructive analysis.

Network configuration verification and attack graphs. Batfish [27] and Minesweeper [6] build formal models of network configuration and verify reachability and policy properties over them, and Batfish in particular is often described as constructing a network digital twin. This work is the direct ancestor of the

logic-based invariants of Thrust 2, and we adopt its property-checking methodology. The difference lies in what the model contains, as these tools model the forwarding plane symbolically, whereas RedTwin instantiates *running software* at the recorded versions and configurations and executes real attacks against it. A symbolic model establishes that a host is reachable; it cannot establish that reaching it yields code execution through a defect in the service running there. Similarly, attack graphs formalize how atomic exploits compose into breaches [61], and MulVAL [45] made the approach practical at deployment scale, but the classical work reasons over a *database of known* exploits and over assumed host configurations, so the resulting graph inherits any error in the inventory it is given. RedTwin closes that loop by deriving the inventory automatically from observation and by discovering new primitives through the analysis of the twin.

Discovery, testbeds, and LLM agents for security. ZMap [25] and ZGrab [74] established fast stateless scanning, osquery [44] exposes host state as queryable tables, and cartography [10] consolidates infrastructure assets into a graph database, closely related to the structured layer of Thrust 1. These tools answer the question of what exists, but none constructs a replica representative from a security perspective, and none treats the load a probe imposes on a fragile host as a first-class constraint. Emulation testbeds such as DETER [8], and our own work on GATE [1], are authored by hand, whereas the twins of RedTwin are derived automatically from a specific deployment. Finally, LLM agents have advanced quickly on offensive tasks [22, 75, 28], but that work evaluates agents on targets that are *given*, typically single hosts or CTF challenges, whereas RedTwin addresses the problem those agents presuppose, that is, the construction of the target, at infrastructure scale, and faithfully enough that what the agent finds is true of a real system.

10 Management Plan and Qualifications of the PIs

Organization. The proposed research is a collaboration between UC Santa Barbara, the lead organization, and Arizona State University, a subawardee. The PIs at UCSB are Giovanni Vigna, the lead PI, and Christopher Kruegel, and the PIs at ASU are Adam Doupé, who coordinates the subaward, and Tiffany Bao. Thrust 1 is led at UCSB, drawing on the service-dependency and network-measurement expertise of PIs Vigna and Kruegel. Thrust 2 is shared, with UCSB leading the replication and fidelity analysis and ASU the firmware re-hosting and mock synthesis, and Thrust 3 is led at ASU for the component-level analysis and at UCSB for the multi-step planning. The upstream engagement with the anchor projects is coordinated by the lead PI. The team holds a weekly all-hands meeting, a monthly thrust-level review, and an annual in-person meeting that rotates between the campuses, and the students at both institutions share a single repository and continuous integration system, as they already do on existing joint efforts.

Qualifications. The four PIs have a long history of joint research and share a hacking team, Shellphish, which gives the team the combined expertise in infrastructure security, program analysis, and autonomous vulnerability discovery that this project requires. All four have contributed to Shellphish's effort in the DARPA Artificial Intelligence Cyber Challenge, collaborating on an autonomous cyber reasoning system that discovers and remediates vulnerabilities in complex open-source software, the most direct precedent for the agentic analysis proposed here.

PIs Kruegel and Vigna have a 20-year-long history of collaboration, and they have developed novel approaches to vulnerability analysis and malware detection, whose results include open-source tools, such as Anubis and angr, and hundreds of papers in security and privacy. Both PIs have substantial experience in managing large grants from NSF, DARPA, and other agencies, including the ACTION AI Institute, and they co-lead a large lab with dozens of students. Their previous work on network critical-service detection [72] and on AI-planning-based privilege escalation [46] underpins Thrusts 1 and 3. PI Doupé's research focuses on automated vulnerability analysis, web security, cybercrime, and cybersecurity education, and PI Bao's examines the broader lifecycle of software vulnerabilities, including those in cyber-physical systems, which bears directly on the fragile-device constraints that shape Thrust 1. PIs Doupé and Bao have also collaboratively created efficient hardware digital twins for vulnerability research, which equips the team with experience directly relevant to Thrust 2.

11 Broader Impacts

Education and mentoring. The PIs have a substantial record in education and curriculum development. PIs Kruegel and Vigna teach several classes related to the proposed research, and these courses are routinely among the highest-ranked in their departments. For instance, PI Vigna received the UCSB Academic Senate Distinguished Teaching Award, the highest teaching award at UC Santa Barbara, and in 2025 PI Kruegel received the UCSB Outstanding Graduate Mentor Award. PIs Doupé and Bao co-teach ASU's undergraduate "Introduction to Cybersecurity" course, required for all Computer Science students and averaging over 700 students a semester. The PIs plan to introduce modules focused explicitly on infrastructure security into their classes, including lab exercises where students find and exploit defects in complex infrastructure and propose solutions to the flaws they identify.

PI Doupé is a lead developer of the online educational platform pwn.college and, together with PI Bao, an Associate Director of ASU's American Cybersecurity Education Institute, which runs the publicly available pwn.college instance and facilitates classes for over 20 colleges around the United States [39, 62, 38]. pwn.college is an open-source system where anyone can learn hands-on cybersecurity material with only a browser, and the PIs plan to create open-source and reusable modules for it that focus on infrastructure security, giving the educational output of this project the same distribution as the platform itself.

The PIs also have a history of including undergraduate and high school students in their research, with students often listed as co-authors on publications. For example, Audrey Dutcher, Paul Grosen, and Jessie Grosen each joined the lab as high school interns and later continued as undergraduates at UCSB and ASU. In addition, PIs Kruegel and Vigna have been faculty mentors in the Early Research Scholars Program, an NSF-supported research apprenticeship that supports students in their first research experience, and a letter of collaboration from Ziad Matni, who leads the program, is attached to this proposal. The PIs also run a summer internship program that has hosted more than 100 undergraduates over the past decade.

Community outreach: the twinCTF competition. The PIs have created the first distributed educational capture-the-flag competition, called the iCTF. The iCTF has run every year since 2003, and in 2026 it introduced a new type of challenge that required participants to develop agents to solve security problems. As part of this project, the PIs will organize an annual public competition, called twinCTF, centered on digital twins. In this competition, the participants will be given meta-information bundles that describe computer infrastructures based on open-source software, and they will need to develop digital twins that represent those infrastructures. The organizers, in turn, will run attacks known to work against the original but localized on the submitted twin, and if an attack succeeds, the participants get points for having created a representative twin. The challenges are drawn from the reference networks of Section 7 and are released publicly afterward, so that the competition also grows the benchmark. twinCTF will be open to the public, and its low barrier to entry and range of difficulty are designed to include students with different backgrounds and skill levels.

Technology transfer, open source, and dissemination. The PIs have a long history of making their research artifacts publicly available, including Anubis [5], Wepawet [14], and angr [3, 63], a binary analysis framework with more than 9K stars on GitHub. The PIs plan to make RedTwin, the reference-network benchmark, and the twinCTF infrastructure available as open source under the process described in Section 6, and to contribute the improvements to the anchor products upstream, so that they reach users who never run RedTwin at all. The PIs also have a successful track record of working with industry partners, and PIs Kruegel and Vigna founded Lastline, a network security company acquired by VMware in 2020. Our results will be published in top-tier security venues, such as the IEEE Symposium on Security and Privacy, the ACM Conference on Computer and Communications Security, the USENIX Security Symposium, and the Network and Distributed Systems Security Symposium.

12 Results from Prior NSF Support

AI Institute for Agent-based Cyber Threat Intelligence and Operation (ACTION). IIS-2228976, $21,994,201, 2023–2028. PIs: G. Vigna and C. Kruegel et al. *Intellectual merit:* using novel knowledge-modeling and logic reasoning to support collaboration among intelligent security agents, ACTION is developing the breakthroughs needed for an agent-driven autonomous process that continuously improves security.

Broader impacts: increased ability to detect attacks and respond to breaches at scale, and support for a large cohort of student researchers. *Publications:* representative results include [26, 46, 2, 29, 47]. *Products:* the institute maintains the Global Accessible Test Infrastructure (GATE), emulated topologies representing smart cities, chemical plants, water systems, and manufacturing plants, available to the research community [1]. *Relation to this proposal:* ACTION develops *defensive* agent collaboration for threat intelligence and response on live systems, whereas RedTwin performs *offensive* assessment on constructed replicas. There is no overlap in scope or deliverables, and the relationship is one of leverage, as the GATE topologies serve as input reference infrastructures for Section 7.

SaTC: CORE: Medium: Bedrock: Security for Smart Contracts. CNS-2334709, $1,200,000, 2024–2027. PIs: C. Kruegel and G. Vigna. *Intellectual merit:* the first multi-chain program analysis platform operating directly on low-level smart-contract bytecode, letting analyses be reused across blockchain environments. *Broader impacts:* protection of Web3 applications, where attacks on smart contracts pose a significant threat to users. *Publications:* [56, 64, 58, 57, 37]. *Products:* Open-source smart contract security analyzers, including the SAILFISH static analysis framework and EVM storage-collision detection tools, released through the investigators' public repositories. *Relation to this proposal:* no overlap.

SaTC: CORE: Medium: Collaborative: Taming Web Content Through Automated Reduction in Browser Functionality. CNS-1703375, $406,000, 2017–2021. PIs: A. Doupé et al. *Intellectual merit:* minimized the browser attack surface by letting website operators limit available JavaScript functionality. *Broader impacts:* uncovered extensions leaking personally identifiable information and produced two defenses reducing extension fingerprintability. *Publications:* [11, 7, 67, 69, 32]. *Products:* prototypes and measurement data released through the project websites and the investigators' public repositories. *Relation to this proposal:* no overlap.

SaTC: CORE: Medium: Symbolizing Viability: Paving the Road to Practical Symbolic Execution. CNS-2247954, $1,199,992, 2023–2027. PIs: T. Bao et al. *Intellectual merit:* improves the usability of symbolic execution. *Broader impacts:* a human-interactive framework lowering the expertise symbolic analysis requires. *Publications:* [34, 73, 53, 36, 23]. *Products:* symbolic execution component under angr-management, a binary analysis platform. *Relation to this proposal:* no overlap; RedTwin uses symbolic execution only as one of several off-the-shelf component-level analyses.

CAREER: Achieving Autonomous Symbolic Execution through Learning from Humans. CNS-2442984, $499,609, 2025–2030. PI: T. Bao. *Intellectual merit:* enables autonomous symbolic execution by learning analysis strategies from human analysts. *Broader impacts:* an AI-enabled autonomous symbolic execution framework and an associated educational program. *Publications:* [34, 73]. *Products:* No products were produced under this award. *Relation to this proposal:* no overlap.

References

[1] The NSF AI Institute for Agent-based Cyber Threat Intelligence and Operation (ACTION). https://action.ucsb.edu, 2026.

[2] Hojjat Aghakhani, Wei Dai, Andre Manoel, Xavier Fernandes, Anant Kharkar, Christopher Kruegel, Giovanni Vigna, David Evans, Benjamin Zorn, and Robert Sim. TrojanPuzzle: Covertly poisoning code-suggestion models. In *IEEE Symposium on Security and Privacy (S&P)*, 2024.

[3] angr. https://angr.io, 2026.

[4] Ansible. https://www.redhat.com/en/ansible-collaborative, 2026.

[5] Ulrich Bayer, Christopher Kruegel, and Engin Kirda. TTAnalyze: A Tool for Analyzing Malware, 2006.

[6] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. A general approach to network configuration verification. In *Proceedings of the ACM SIGCOMM Conference*, 2017.

[7] Aidan Beggs and Alexandros Kapravelos. Wild Extensions: Discovering and Analyzing Unlisted Chrome Extensions. In *Proceedings of the Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, June 2019.

[8] Terry Benzel. The science of cyber security experimentation: The DETER project. *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*, 2011.

[9] MITRE CALDERA: A Scalable, Automated Adversary Emulation Platform. https://caldera.mitre.org/, 2026.

[10] Cartography: A Tool that Consolidates Infrastructure Assets into a Graph Database. https://github.com/cartography-cncf/cartography, 2026.

[11] Quan Chen and Alexandros Kapravelos. Mystique: Uncovering information leakage from browser extensions. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2018.

[12] Securing the Software Supply Chain: Recommended Practices Guide for Developers. U.S. Cybersecurity and Infrastructure Security Agency and U.S. National Security Agency, 2022.

[13] Abraham Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. HALucinator: Firmware Re-hosting through Abstraction Layer Emulation. In *Proceedings of the USENIX Security Symposium*, Boston, MA, August 2020.

[14] Marco Cova, Christopher Kruegel, and Giovanni Vigna. Detection and Analysis of Drive-by-Download Attacks and Malicious JavaScript Code. In *Proceedings of the World Wide Web Conference (WWW)*, Raleigh, NC, April 2010.

[15] CVE-2013-2028: nginx stack-based buffer overflow via chunked transfer encoding. https://nvd.nist.gov/vuln/detail/CVE-2013-2028, 2013.

[16] CVE-2019-20372: nginx request smuggling via error_page. https://nvd.nist.gov/vuln/detail/CVE-2019-20372, 2019.

[17] CVE-2021-23017: nginx resolver off-by-one heap write. https://nvd.nist.gov/vuln/detail/CVE-2021-23017, 2021.

[18] CVE-2021-44228 (“Log4Shell”): remote code execution in Apache Log4j. https://nvd.nist.gov/vuln/detail/CVE-2021-44228, 2021.

[19] CVE-2022-41742: nginx ngx_http_mp4_module memory disclosure and crash. https://nvd.nist.gov/vuln/detail/CVE-2022-41742, 2022.

[20] CVE-2024-3094: malicious code introduced by a maintainer into xz-utils. https://nvd.nist.gov/vuln/detail/CVE-2024-3094, 2024.

[21] CVE-2025-1974 ("IngressNightmare"): unauthenticated remote code execution in the ingress-nginx admission controller. https://nvd.nist.gov/vuln/detail/CVE-2025-1974, 2025.

[22] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the USENIX Security Symposium*, 2024.

[23] Fangzhou Dong, Arvind S. Raj, Efrén López-Morales, Siyu Liu, Yan Shoshitaishvili, Tiffany Bao, Adam Doupé, Muslum Ozgur Ozmen, and Ruoyu Wang. Discovering blind-trust vulnerabilities in plc binaries via state machine recovery. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium 2026*, 2026.

[24] Adam Doupé and Zachary Smith. Universal Theory of Exploitation: Synthesizing Weird Machine Programs from Guarded Behavior Models. https://adamdoupe.com/publications/ute-tech-report.pdf, 2026.

[25] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. ZMap: Fast internet-wide scanning and its security applications. In *Proceedings of the USENIX Security Symposium*, Washington, D.C., August 2013.

[26] Marius Fleischer, Dipanjan Das, Priyanka Bose, Weiheng Bai, Kangjie Lu, Mathias Payer, Christopher Kruegel, and Giovanni Vigna. ACTOR: Action-Guided Kernel Fuzzing. In *Proceedings of the USENIX Security Symposium*, Los Angeles, USA, August 2023.

[27] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. A general approach to network configuration analysis. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015.

[28] Linus Folkerts, Will Payne, Simon Inman, Philippos Giavridis, Joe Skinner, Sam Deverett, James Aung, Ekin Zorer, Michael Schmatz, Mahmoud Ghanem, John Wilkinson, Alan Steer, Vy Hong, and Jessica Wang. Measuring AI Agents' Progress on Multi-Step Cyber Attack Scenarios. https://arxiv.org/abs/2603.11214, 2026.

[29] Wil Gibbs, Aravind Raj, Jayakrishna Menon Vadayath, Hui Jun Tay, Jack Miller, Arvind Ajayan, Zion Basque, Audrey Dutcher, Fan Dong, Xander Maso, Giovanni Vigna, Christopher Kruegel, Adam Doupé, Yan Shoshitaishvili, and Ruoyu Wang. Operation mango: Scalable discovery of taint-style vulnerabilities in binary firmware services. In *Proceedings of the USENIX Security Symposium*, 2024.

[30] Eric Gustafson, Marius Muench, Chad Spensky, Nilo Redini, Aravind Machiry, Aurelien Francillon, Davide Balzarotti, Yung Ryn Choe, Christopher Kruegel, and Giovanni Vigna. Toward the Analysis of Embedded Firmware Through Automated Re-hosting. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, Beijing, China, September 2019. USENIX Association.

[31] Istio. https://istio.io/, 2026.

[32] Rasoul Jahanshahi, Adam Doupé, and Manuel Egele. You shall not pass: Mitigating SQL Injection Attacks on Legacy Web Applications. In *Proceedings of the ACM Symposium on Information, Computer and Communications Security (AsiaCCS)*, 2020.

[33] Keycloak: Open Source Identity and Access Management. https://www.keycloak.org/, 2026.

[34] Yi Jou Li, Zeming Yu, James A Mattei, Ananta Soneji, Zhibo Sun, Ruoyu Wang, Jaron Mink, Daniel Votipka, and Tiffany Bao. I can se clearly now: Investigating the effectiveness of gui-based symbolic

execution for software vulnerability discovery. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, CHI '26, New York, NY, USA, 2026. Association for Computing Machinery.

[35] Aravind Machiry, Chad Spensky, Jake Corina, Nick Stephens, Christopher Kruegel, and Giovanni Vigna. DR.CHECKER: A Soundy Analysis for Linux Kernel Drivers. In *Proceedings of the USENIX Security Symposium*, Vancouver, BC, August 2017.

[36] James Mattei, Andrew Lin, Jasper Geer, Jie Hu, Moritz Schloegel, Tiffany Bao, and Daniel Votipka. SoK: A Modularized Framework for Symbolic Execution and Application for Usable Tool Design. In *Proceedings of the 2026 ACM Secure Development Conference (SecDev '26)*, pages 57–73, 2026.

[37] Dongyu Meng, Fabio Gritti, Robert McLaughlin, Nicola Ruaro, Ilya Grishchenko, Christopher Kruegel, and Giovanni Vigna. HOUSTON: Real-time anomaly detection of attacks against Ethereum DeFi protocols. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026.

[38] Connor Nelson, Adam Doupé, and Yan Shoshitaishvili. Sensai: Large language models as applied cybersecurity tutors. In *Proceedings of the Technical Symposium on Computer Science Education (SIGCSE)*, February 2025.

[39] Connor Nelson, Robert Wasinger, Adam Doupé, and Yan Shoshitaishvili. Open Cybersecurity Education: Five Years of pwn.college. In *Proceedings of the Technical Symposium on Computer Science Education (SIGCSE)*, February 2026.

[40] Neo4j. https://neo4j.com/, 2026.

[41] nginx. https://nginx.org/, 2026.

[42] nginx source repository. https://github.com/nginx/nginx, 2026.

[43] Open Source Security Foundation Best Practices Badge Program. https://www.bestpractices.dev/, 2026.

[44] osquery: SQL-powered Operating System Instrumentation and Analytics. https://osquery.io/, 2026.

[45] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. MulVAL: A logic-based network security analyzer. In *Proceedings of the USENIX Security Symposium*, 2005.

[46] Giulio Pasquale, Ilya Grishchenko, Riccardo Iesari, Gabriel Pizarro, Lorenzo Cavallaro, Christopher Kruegel, and Giovanni Vigna. ChainReactor: Automated privilege escalation chain discovery via AI planning. In *Proceedings of the USENIX Security Symposium*, Philadelphia, PA, August 2024.

[47] Stijn Pletinckx, Christopher Kruegel, and Giovanni Vigna. A large-scale measurement study of the PROXY protocol and its security implications. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[48] PostgreSQL: The World's Most Advanced Open Source Relational Database. https://www.postgresql.org/, 2026.

[49] puppet. https://www.puppet.com/, 2026.

[50] Qdrant: High-Performance Vector Search at Scale. https://qdrant.tech/, 2026.

[51] Qemu. https://www.qemu.org/, 2026.

[52] RabbitMQ: One broker to queue them all. https://www.rabbitmq.com/, 2026.

[53] Arvind S Raj, Wil Gibbs, Fangzhou Dong, Jayakrishna Menon Vadayath, Michael Tompkins, Steven Wirsz, Yibo Liu, Zhenghao Hu, Chang Zhu, Gokulkrishna Praveen Menon, Brendan Dolan-Gavitt, Adam Doupé, Ruoyu Wang, Yan Shoshitaishvili, and Tiffany Bao. *Fuzz to the Future:* Uncovering

Occluded Future Vulnerabilities via Robust Fuzzing. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, 2024.

[54] Nilo Redini, Aravind Machiry, Ruoyu Wang, Chad Spensky, Andrea Continella, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. KARONTE: Detecting Insecure Multi-binary Interactions in Embedded Firmware. In *Proceedings of the IEEE Symposium on Security and Privacy*, San Francisco, CA, May 2020.

[55] Nicola Ruaro, Lukas Dresel, Kyle Zeng, Tiffany Bao, Mario Polino, Andrea Continella, Stefano Zanero, Christopher Kruegel, and Giovanni Vigna. SyML: Guiding Symbolic Execution Toward Vulnerable States Through Pattern Learning. In *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, San Sebastian, Spain, October 2021.

[56] Nicola Ruaro, Fabio Gritti, Robert McLaughlin, Ilya Grishchenko, Christopher Kruegel, and Giovanni Vigna. Not your Type! Detecting Storage Collision Vulnerabilities in Ethereum Smart Contracts. In *Proceedings of the Network and Distributed Systems Security Symposium (NDSS)*, San Diego, USA, February 2024.

[57] Nicola Ruaro, Fabio Gritti, Robert McLaughlin, Dongyu Meng, Ilya Grishchenko, Christopher Kruegel, and Giovanni Vigna. A History of Greed: Practical Symbolic Execution for Ethereum Smart Contracts. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, pages 275–296, Graz, Austria, July 2025. Springer.

[58] Nicola Ruaro, Fabio Gritti, Dongyu Meng, Robert McLaughlin, Ilya Grishchenko, Christopher Kruegel, and Giovanni Vigna. Approve Once, Regret Forever: On the Exploitation of Ethereum's Approve-TransferFrom Ecosystem. In *Proceedings of the USENIX Security Symposium*, Seattle, WA, August 2025.

[59] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, Christopher Kruegel, Thorsten Holz, and Ali Abbasi. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In *Proceedings of the USENIX Security Symposium*, Boston, USA, August 2022.

[60] Shellphish. Artiphishell: An agentic cyber reasoning system for the DARPA AIxCC competition. https://support.shellphish.net/aixcc/team/, 2024.

[61] Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette Wing. Automated generation and analysis of attack graphs. In *Proceedings of the IEEE Symposium on Research in Security and Privacy*, 2002.

[62] Yan Shoshitaishvili, Adam Doupé, and Connor Nelson. The Linux Luminarium: Learning Linux by Leveraging Lightweight Labs and Ludicrous Lessons. In *Proceedings of the Technical Symposium on Computer Science Education (SIGCSE)*, February 2026.

[63] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Audrey Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis. In *IEEE Symposium on Security and Privacy*, 2016.

[64] Ravindu De Silva, Wenbo Guo, Nicola Ruaro, Ilya Grishchenko, Christopher Kruegel, and Giovanni Vigna. GuideEnricher: Protecting the Anonymity of Ethereum Mixing Service Users with Deep Reinforcement Learning. In *Proceedings of the USENIX Security Symposium*, Philadelphia, PA, August 2024.

[65] SLSA: Supply-chain Levels for Software Artifacts. https://slsa.dev/, 2026.

[66] Chad Spensky, Aravind Machiry, Nilo Redini, Colin Unger, Graham Foster, Evan Blasband, Hamed Okhravi, Christopher Kruegel, and Giovanni Vigna. Conware: Automated Modeling of Hardware

Peripherals. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, Hong Kong, China, June 2021.

[67] Oleksii Starov, Pierre Laperdrix, Alexandros Kapravelos, and Nick Nikiforakis. Unnecessarily Identifiable: Quantifying the fingerprintability of browser extensions due to bloat. In *Proceedings of The Web Conference*, May 2019.

[68] Mattia Tantardini, Francesca Ieva, Lucia Tajoli, and Carlo Piccardi. Comparing methods for comparing networks. *Scientific Reports*, 9(1):17557, 2019.

[69] Erik Trickel, Oleksii Starov, Alexandros Kapravelos, Nick Nikiforakis, and Adam Doupe. Everyone is Different: Client-side Diversification for Defending Against Extension Fingerprinting. In *Proceedings of the USENIX Security Symposium*, August 2019.

[70] Orange Tsai. Breaking Parser Logic: Take Your Path Normalization Off and Pop 0days Out. In *Black Hat USA*, 2018.

[71] Weaviate. https://weaviate.io/, 2026.

[72] Ali Zand, Amir Houmansadr, Giovanni Vigna, Richard Kemmerer, and Christopher Kruegel. Know your achilles' heel: Automatic detection of network critical services. In *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*, 2015.

[73] Kyle Zeng, Moritz Schloegel, Christopher Salls, Adam Doupé, Ruoyu Wang, Yan Shoshitaishvili, and Tiffany Bao. Ropbot: Reimagining code reuse attack synthesis. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*. Internet Society, February 2026.

[74] ZGrab 2.0: Application-Layer Scanner. https://github.com/zmap/zgrab2, 2026.

[75] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[76] The ZMap Project: ZMap network scanner. https://github.com/zmap/zmap, 2026.